

THE UNLOX ACADEMY

Data Analytics & Data Science Track

BingePlay

Streaming Analytics Project

Week 4 · Advanced SQL · Minor Project

Duration: 7 days

BingePlay is a fictional Indian OTT streaming service. You're joining their newly formed analytics team. This project takes you through 12 real business questions your product manager might ask on day one - from simple revenue reports to advanced churn detection.

1. Scenario

You've just joined BingePlay as their first data analyst. BingePlay launched in India on 1 January 2024 with three subscription plans (Basic ₹199, Premium ₹399, Family ₹699/month), a growing catalog of 100 shows across 7 languages, and roughly 3,000 users to date.

Your Head of Product hands you access to the production database. She wants answers to 12 business questions - a mix of fundamentals (Tier 1), day-to-day analytics (Tier 2), and advanced product intelligence (Tier 3). Everything below is real business framing. Nothing is fabricated for exercise value.

Some questions are traps designed to catch analysts who don't think carefully about NULLs, joins, or window boundaries. Read every question twice before writing SQL.

BUSINESS CONTEXT

Why streaming analytics is a great learning ground.

Streaming platforms are the perfect SQL playground - every user does dozens of things (browse, watch, rate, cancel, renew), and the data creates rich patterns to query.

Analytics teams at Hotstar, Netflix, Prime Video use variations of these exact questions daily.

Retention analysis, binge detection, upsell tracking, real work.

2. Database Schema

Five tables. Study the columns carefully, several questions depend on details you might miss on a fast read (like the min_plan column, or the fact that user_id in watch_sessions is nullable).

2.1 users - 3,000 rows

Column	Type	Description
user_id	VARCHAR(10) PK	Format: U00001 to U03000
Name	VARCHAR	Full name
signup_date	DATE	When they signed up (Jan-Jun 2024)
city	VARCHAR	One of 10 Indian cities
age_group	VARCHAR	18-24, 25-34, 35-44, 45-54, 55+
referral_source	VARCHAR	organic, social_media, friend, ads, partner

2.2 subscriptions - 4,497 rows

Column	Type	Description
subscription_id	VARCHAR(15) PK	Format: SUB00001...
user_id	VARCHAR(10) FK	One user can have multiple sub records
Plan	VARCHAR	Basic / Premium / Family
start_date	DATE	When the subscription began
end_date	DATE (NULL)	NULL means still ongoing (open-ended)
status	VARCHAR	active / expired / cancelled
monthly_price_inr	INT	Basic=199, Premium=399, Family=699

WARNING

The subscription table has multiple rows per user.

One user can have multiple subscription records - from renewals, upgrades, downgrades, or churn-and-return. A user's history is a SEQUENCE of subscription rows sorted by start_date.

"active" status alone does NOT mean the sub is currently open. An 'active' sub with end_date > TODAY is currently open. An 'active' sub with end_date <= TODAY is no longer open.

2.3 shows - 100 rows

Column	Type	Description
show_id	VARCHAR(10) PK	Format: S001 to S100
title	VARCHAR	Show title
category	VARCHAR	Drama, Comedy, Thriller, Action, Romance, Documentary, Kids, Reality
language	VARCHAR	Hindi, English, Tamil, Telugu, Malayalam, Kannada, Bengali
release_year	INT	2018-2024
imdb_rating	DECIMAL(3,1)	4.0-9.5
is_original	TINYINT	1 = BingePlay Original, 0 = acquired content
min_plan	VARCHAR	Basic / Premium / Family - minimum plan required to watch

Important - min_plan: some shows are Basic-tier (anyone with any plan can watch), some require Premium+, and a few need Family plan. This matters for Q8.

2.4 watch_sessions - 100,351 rows

Column	Type	Description
session_id	INT PK	1..100351
user_id	VARCHAR(10) NULLABLE	The user watching. **Yes - this is nullable.** See Q7.
show_id	VARCHAR(10) FK	Which show
session_date	DATE	The date of the session (Jan-Jun 2024)
watch_minutes	INT	How long they watched (1-180 min)
device_type	VARCHAR	Mobile / TV / Laptop / Tablet
completed	TINYINT	1 if the session ended "complete" - user watched to end

WARNING

watch_sessions.user_id has NULL values.

There are exactly 2 rows in watch_sessions where user_id IS NULL. This reflects real-world data quality issues (e.g., logging bug, guest previews).

Column	Type	Description
This detail matters for Q7. A naive `NOT IN (SELECT user_id FROM watch_sessions)` on nullable columns silently returns ZERO rows in SQL - because NOT IN with NULL evaluates to UNKNOWN. Use IS NULL logic or NOT EXISTS. This is a classic interview trap. Handle it correctly = 8 marks. Miss it = 0 marks on Q7.		

2.5 ratings - 5,000 rows

Column	Type	Description
rating_id	INT PK	1..5000
user_id	VARCHAR(10) FK	Who rated
show_id	VARCHAR(10) FK	What they rated
stars	TINYINT	1 to 5
rated_date	DATE	When rated

3. Setup

Load the database

1. Ensure MySQL is running on your machine (any recent version).
2. Open MySQL Workbench or your CLI client.
3. Run the setup file:

```
-- In MySQL Workbench:  
-- File → Open SQL Script → bingeplay_setup.sql → Execute (Ctrl+Shift+Enter)  
  
-- Or via CLI:  
mysql -u root -p < bingeplay_setup.sql
```

4. Confirm the row counts:

```
USE bingeplay;  
SELECT 'users' AS tbl, COUNT(*) FROM users  
UNION ALL SELECT 'subscriptions', COUNT(*) FROM subscriptions  
UNION ALL SELECT 'shows', COUNT(*) FROM shows  
UNION ALL SELECT 'watch_sessions', COUNT(*) FROM watch_sessions  
UNION ALL SELECT 'ratings', COUNT(*) FROM ratings;  
  
-- Expected output:  
--   users          3000  
--   subscriptions  4497  
--   shows           100  
--   watch_sessions 100351  
--   ratings         5000
```

Verify the NULL trap

```
SELECT COUNT(*) FROM watch_sessions WHERE user_id IS NULL;  
-- Expected: 2
```

4. Scoring Rubric

Tier	Skills	# Q's	Marks each	Subtotal
Tier 1 - Foundations	SELECT, WHERE, GROUP BY, aggregates, NULL handling	5	6	30
Tier 2 - Joins & Subqueries	JOINS, subqueries, NOT EXISTS, self-joins	5	8	40
Tier 3 - Advanced	Window functions, gaps-and-islands, LAG	2	10	20
Code quality	Aliases, formatting, meaningful CTEs	-	-	10
TOTAL		12		100

Grade bands

- **0-59:** Fail - retake required
- **60-74:** Pass
- **75-84:** Merit
- **85+:** Distinction

How each question is graded

For each question:

- Correct numeric answer: 60% of the marks
- SQL query provided (correct approach): 30% of the marks
- Interpretation / explanation (where asked): 10% of the marks

5. Tier 1 - Foundations

Q1 - Active revenue

As of 30 June 2024, compute the total monthly recurring revenue from all ACTIVE subscriptions.

A subscription is "active" for this purpose if:

- status = 'active', AND
- end_date IS NULL, OR end_date > '2024-06-30'

Deliverables:

- Count of active subscriptions
- Total monthly revenue in ₹

HINT**The NULL catch.**

Forgetting the end_date IS NULL check drops all currently-open subscriptions (end_date not yet set). That would silently under-report by ~40%.

Q2 - Signup momentum

For each calendar month in 2024, count the number of new user signups.

Deliverables:

- Table with columns (month, signup_count) for months 1 through 6
- Name of the month with the highest signups

Q3 - Device analytics

For each device_type in watch_sessions, compute:

- Total number of sessions
- Total watch_minutes across all sessions
- Average watch_minutes per session
- Completion rate - % of sessions where completed = 1

Note: exclude NULL user_id sessions from your count. Round completion rate to 2 decimal places.

Q4 - Rating distribution

Show how ratings are distributed:

- Count and percentage of ratings for each star value (1 to 5)
- What percentage of all ratings are 4 or 5 stars?

Round percentages to 2 decimal places.

Q5 - Originals vs acquired

Compare BingePlay Originals (is_original = 1) against acquired content (is_original = 0).

Compute for each group:

- Number of shows
- Average imdb_rating (2 decimal places)

- Average release_year

Then answer in 1-2 sentences: which group performs better on ratings, and by how much?

6. Tier 2 - Joins & Subqueries

Q6 - Binge day detection

A "binge day" is defined as: a single user watching the same show 5 or more times on the same calendar date.

For Q2 2024 (1 April – 30 June 2024) specifically:

- Total number of binge days that occurred (across all users and shows)
- The user_id who had the MOST binge days in Q2 2024
- How many binge days that user had

HINT

Careful grouping.

The definition is (user + show + date) → 5+ sessions. If you group by only (user + date) you'll wrongly count "user watched 5 sessions across 3 different shows" as a binge day.

Q7 - Q1 signups who never watched (THE NULL TRAP)

Find users who:

- Signed up in Q1 2024 (January, February, or March), AND
- Have never watched a single session (no rows in watch_sessions with their user_id)

Deliverables:

- Total number of Q1 signups
- Number of Q1 signups who have never watched anything

WARNING

This is the interview classic.

If you write: WHERE user_id NOT IN (SELECT user_id FROM watch_sessions) - your query returns 0 rows. Silently. Because NOT IN evaluates to UNKNOWN when the subquery contains NULLs, and UNKNOWN is treated as false.

Correct approaches: LEFT JOIN + IS NULL, or NOT EXISTS with an appropriate correlated subquery.

Solving this correctly demonstrates real SQL maturity. Every senior interviewer tests for it.

Q8 - The over-paying Premium/Family users

Some users are on Premium or Family plans but ONLY ever watch shows that were available on the Basic plan (min_plan = 'Basic'). They're over-paying and are prime targets for a downgrade offer.

Find these users. Consider a user's CURRENT plan as of 30-Jun-2024 (from their most recent active subscription).

Deliverables:

- Number of Premium/Family users whose entire watch history is Basic-tier only

HINT

Structure this as an anti-existence check.

For each Premium/Family user, you want the case where NO show they watched has min_plan IN ('Premium', 'Family'). NOT EXISTS with a correlated subquery on the negation is the natural pattern.

Q9 - Upgrade success cohort

Find users who:

- Signed up in January 2024
- Started with a Basic plan (their earliest subscription record had plan = 'Basic')
- At some point upgraded to Premium or Family (a later subscription record with a higher plan)
- Are still active as of 30-Jun-2024

Deliverables:

- Number of such users
- Their average days from signup to first upgrade

HINT

Multiple subscription rows per user.

You'll need to think about what "first" and "upgrade" mean when a user has many subscription rows. ORDER BY start_date within user + ROW_NUMBER() is one clean approach.

Q10 - Cliffhanger comebacks

A "cliffhanger comeback" is: a user has an INCOMPLETE session (completed = 0) of some show, then within 1 to 7 days after, watches the SAME show again.

Deliverables:

- Total number of cliffhanger comeback EVENTS (count each unique (user, show, incomplete_date) once)
- The show_id that generates the most cliffhanger comebacks
- The title of that show

HINT**Self-join on watch_sessions.**

Join watch_sessions to itself with the conditions: same user_id, same show_id, first session completed = 0, second session's date is between (first.session_date + 1) and (first.session_date + 7).

7. Tier 3 - The Killer Questions

Q11 - Consecutive-week engagement

Find users who watched at least ONE session in FOUR or more CONSECUTIVE calendar weeks. Use ISO week numbering.

Deliverables:

- Number of users with a streak of 4+ consecutive weeks
- The longest streak in weeks (across all users)
- One user_id who has that longest streak

HINT

This is the classic gaps-and-islands problem.

The standard pattern: get distinct (user_id, week_key) rows, then use ROW_NUMBER() within each user ordered by week. Subtract that row_number from a numeric week ordinal. Rows in the SAME consecutive streak have the same difference. GROUP BY (user_id, difference) and COUNT to get streak lengths.

This is a genuinely hard SQL problem. Nobody solves it first try. Take your time.

Q12 - Churn signal detection

Find users whose total watch_minutes in June 2024 dropped by 50% or more compared to their total watch_minutes in May 2024. These are early churn signals - the product team can target them with retention offers.

For each such user, return:

- user_id
- name (join to users)
- Total May 2024 watch_minutes
- Total June 2024 watch_minutes
- Drop percentage - rounded to 2 decimals

Also report: the TOTAL number of "churn signal" users.

Only consider users who had at least SOME watch activity in May 2024 (otherwise you can't compute a drop).

HINT

The window-function-in-WHERE trap.

You'll compute monthly aggregates then compare. A common attempt is: `SELECT ..., LAG(month_mins) OVER (PARTITION BY user_id ORDER BY month) AS prev_mins, ..., WHERE (prev_mins - month_mins) / prev_mins >= 0.5.`

This FAILS because you can't reference `LAG()` output directly in a `WHERE` clause of the same query. Wrap it in a CTE first, then filter in the outer query.

This CTE-then-filter pattern is one of the most common needs in real analytics work.

8. What to Submit

Submit ONE Jupyter notebook (.ipynb) named:

```
bingeplay_<yourname>.ipynb
```

The notebook must:

- Connect to your MySQL bingeplay database using SQLAlchemy + pymysql
- Have one code cell per question containing your SQL query executed via `pandas.read_sql`
- Print the query output
- Have one markdown cell per question containing your final numeric answer + brief interpretation where asked

Structure per question:

```
# Markdown cell:
## Q7 – Q1 signups who never watched
**Answer: <number> users**
**Total Q1 signups: <number>**

# Code cell:
query = """
SELECT COUNT(*) AS never_watched
FROM users u
LEFT JOIN watch_sessions ws ON u.user_id = ws.user_id
WHERE MONTH(u.signup_date) <= 3
      AND YEAR(u.signup_date) = 2024
      AND ws.session_id IS NULL;
"""
pd.read_sql(query, engine)
```

Deadline

Submit within 7 days of the project release date.

Anti-cheating notes

- **The dataset is unique to this batch.** The specific answers cannot be found online or guessed by an LLM. If your numbers don't match, your query is wrong.
- **Copying from ChatGPT/Claude is easy to detect** because LLMs will produce fluent-sounding but numerically incorrect answers on this specific dataset.
- **Collaboration and discussion are ENCOURAGED.** But every submission must be your own query and your own writeup.

- **During grading you may be asked to explain your Tier 2 or Tier 3 queries live.** If you can't explain what you wrote, you get 0 on that question - regardless of whether the number is right.

9. Advice From the Mentors

This project is designed to feel like a real analytics job - not a textbook exercise. Some observations from mentors who've graded past cohorts:

- **Start with Tier 1.** Verify your numbers match the expected values before moving on. If Q1-Q5 aren't right, everything downstream compounds.
- **Read questions twice.** Every hint callout in this brief is there because past cohorts fell into that exact trap.
- **Q7 and Q11 are the differentiators.** Q7 tests real SQL maturity (NULL trap). Q11 tests real problem-solving (gaps-and-islands). Distinction candidates get both right.
- **Q12's WHERE-can't-reference-window trap** is one of the most common bugs in production dashboards. Learning to reach for CTEs is a core skill.
- **Don't optimize queries.** Optimize for READABILITY - proper aliases, meaningful CTE names, comments. Fast wrong queries earn 0. Slow correct ones earn full marks.

Good luck.

— The Unlox Academy Team